

M I C R O S O F T   A G E N T   H A C K A T H O N   2 0 2 6

# MOCHI-kiki

Teams 会議に常駐する Agentic AI

会議の場で動く相棒 — 仕様補完と、ライブ議事録ビュー

---

Azure / Semantic Kernel / Recall.ai / Bot Framework

Ruuuhs (@mochitec)

# 会議が止まる、半年後にすれ違う

## 会議中に起きていること

- 用語・仕様を確認するために、誰かが画面共有でドキュメントを探し始める
- 「〇〇って何でしたっけ」に答えられず、議論が止まる
- 「あれをやっておいて」のまま会議が終わり、半年後にすれ違う

## Microsoft 365 Copilot Recap

- 会議後の要約・アクション抽出は十分まかなえる
- ただし「会議が終わってから」動く設計
- 用語不明や仕様確認の場面では、会議中に助けてくれない

# MOCHI-kiki — 会議の場で動く相棒

MVP は 2 機能で構成しています。

C - 02

## ドキュメント参照補完

用語・仕様などドキュメント参照が必要な発言を検知し、社内ドキュメントを引いて 200 字の回答を会議チャットへ自発的に投稿する。

C - 04

## ライブ議事録ビュー

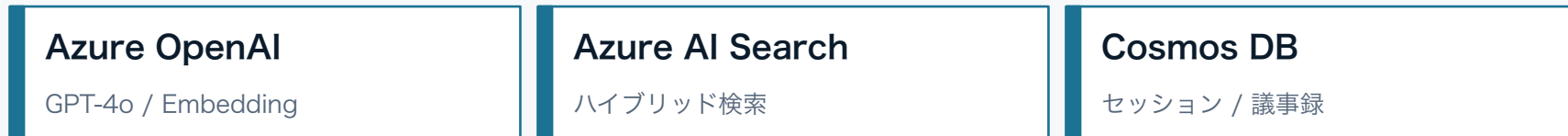
会議の発話から議事録とタイムラインをライブ生成し、管理コンソールで閲覧できる。会議チャットは汚さない。

# アーキテクチャ全体像

Recall.ai が音声と文字起こしを返し、Semantic Kernel オーケストレータが意図検知と RAG を回す。



## バックエンドサービス



# ドキュメント参照補完 — 5 ステップ

|    |                  |                                                             |
|----|------------------|-------------------------------------------------------------|
| 01 | IntentAnalysis   | 発話を分類 (spec_inquiry / ambiguous / normal) し、キーワードを JSON で返す |
| 02 | RAGSearch        | spec_inquiry のキーワードで Azure AI Search のハイブリッド検索 (上位 3 件)     |
| 03 | AnswerGeneration | GPT-4o で 3 件をコンテキストに 200 字以内へ整形                             |
| 04 | Guard            | Python 側で text[:200] と [仕様補完] プレフィックスを二重ガード                 |
| 05 | ChatPoster       | Bot Framework で proactive メッセージとして会議チャットへ投稿                 |

# ドキュメント参照補完

「RU 上限は」と発話 → 数十秒後にチャットへ自動補足

次のスライドで実画面 (4-6 倍速)

## 【 画面録画 A を差し込む 】

Teams 会議 (左) + チャット欄 (右) のスプリット。発話 → 数十秒で [仕様補完] 自動投稿。  
実時間 ~25 秒は 4-6x 倍速 + テロップ「実測 ~25 秒」を重ねる。

# ライブ議事録ビュー — 会議チャットを汚さない

管理コンソール /meeting?id=... で 3 つのテンポを 1 画面に同居させる。

## 議事録

GPT-4o で「議題 / 決定事項 / TODO / 質疑応答」を構造化  
Markdown で全文再生成

更新 90 秒

## タイムライン

5 分ブロックごとに 150 字要約。  
末尾ブロックのみ再要約し、確定  
済みには触らない

更新 90 秒

## 文字起こし

発話データをそのまま表示。最新  
の発話を即座に追いかける

更新 5 秒



## 【 画面録画 B を差し込む 】

管理コンソール /meeting?id=... の 3 ビューを順に映す。  
議事録 → タイムライン → 文字起こし。更新間隔の差をテロップで補足。

## 現状の弱点 — 補完投稿のレイテンシ

発話から投稿まで数十秒。「即答」は MVP では未達。後追い補足として機能している段階。

### 原因

- Recall.ai の日本語 STT は精度モード固定 (prioritize\_accuracy)
- 発話塊が一区切りつくまで貯めてから返すため、STT だけで十数～数十秒
- 日本語では prioritize\_low\_latency が選べない (英語専用)

### 本番投入前の宿題

- Recall.ai を諦め、低遅延な STT に置き換える検証
- Recall.ai だけで現実的な Teams 音声取得と即答は両立しない、というのが MVP の学び

# 使用した Microsoft 技術

## Azure Container Apps

Bot プロセス実行基盤 (必須要件 ①)

## Azure AI Search

RAG ハイブリッド検索

## Azure Cosmos DB

セッション / 議事録の永続化 (autoscale)

## Semantic Kernel

Intent / RAG / Answer プラグイン構成

## Azure OpenAI

GPT-4o + Embedding (必須要件 ②)

## Azure AI Speech

音声処理 (補助)

## Azure Key Vault

シークレット管理

## Bot Framework + Teams

会議チャットへの proactive 投稿

THANK YOU

# 会議の場で動く相棒を、もう少し前へ。

詳細は Zenn の記事をご覧ください。

ARTICLE      [zenn.dev/ruuuhs/articles/mochi-kiki-teams-agent](https://zenn.dev/ruuuhs/articles/mochi-kiki-teams-agent)

HACKATHON      Microsoft Agent Hackathon 2026

GITHUB      [github.com/mochitec/MOCHI-kiki](https://github.com/mochitec/MOCHI-kiki) (公開予定)

Ruuuhs (@mochitec) / MOCHI-tec